Air Quality Analysis and Prediction Using Apache Spark

Submitted by:

Name: Velagala Rohan Sai Kumar Reddy

Registration No: 12311004

Programme and Section: B. Tech CSE, K4P23WR

Course Code: INT315

Roll No: 08

Under the Guidance of

Pardeep Kumar

Discipline of CSE/IT

Lovely School of Computer Science and Engineering

Lovely Professional University, Phagwara

Table of Contents

1. Project Title
2. Problem Statement
3. Objective of the Project
4. Dataset Description
5. Dataset Source
6. Tools & Technologies Used
7. Implementation / Methodology
8. Complete Code
9. Execution Screenshots
10. Output / Result Screenshots
11. Result Analysis
12. Conclusion

1. Project Title

Air Quality Analysis and Prediction Using Apache Spark

2. Problem Statement

Air quality impacts the well-being of humans and other life, as well as the world as a whole, because high concentrations of pollutants in the air have detrimental effects. Air quality datasets track many kinds of air pollutants, the minimum, maximum, and average amounts recorded, and include location data for city, state, latitude, and longitude.

A large dataset like this is a nightmare to manually search through. It's hard to discern trends or even connections between the different air pollutants. To address these problems, I used Apache Spark and PySpark. I implemented some machine learning techniques on this air quality dataset to predict average pollutant levels and to put air quality samples into a number of user defined pollutant categories. Data visualization was used to explore the different air quality parameters and their relationships.

3. Objectives

The primary purpose of this work is the examination of air quality data through the utilization of the Apache Spark big data framework. My goals involve cleaning and transforming the data in order to ready the dataset for machine learning algorithms, along with an exploration of pollutant distribution, linear regression for mean pollutant prediction, and decision tree/random forest for pollutant classification. Further work includes utilizing visualization techniques to discern patterns in data, as well as an exploration of numerical data relationships and cross pollutant comparisons.

4. Dataset Description

For this project, I used an air quality dataset that recorded the levels of several pollutants across different monitoring sites in India. The data set has the geographical location of the site (country, state, city, and site name), latitude, and longitude. It also lists the pollutant's measurements as well as the last update time. The key attributes regarding the pollutants in the dataset are the ID, the minimum value, the maximum value, and the average value. At the beginning, the dataset contained 3,724 records and 11 columns. After transforming the pollutant measurements into numbers, the dataset was left with 3,381 records to be used for machine learning.

5. Dataset Source

In this project, the air quality dataset is collected from the Open Government Data (OGD) Platform of the Government of India. This platform is a source of various datasets on the sectors such as environment monitoring of India. This dataset includes information about air pollutants and their locations.

Source: Government of India – Open Government Data Platform

Dataset Link: https://www.data.gov.in/catalog/real-time-air-quality-index

6. Tools and Technologies Used

- Apache Spark: Cluster Computing Framework used for air quality data processing and analysis.
- PySpark: Used for data loading, cleaning, preprocessing, feature engineering and machine learning processes.
- Python: Programming language used in the project.
- Spark MLlib: Used for developing Linear Regression, Decision Tree, and Random Forest models.
- Pandas: Used for presenting the chosen aggregate data in tabular form.
- Matplotlib: Used to generate and save bar charts, scatter charts, box charts, and correlation charts.
- Dataset: Air Quality Dataset taken from the Open Government Data Platform of the Government of India.
- Development Environment: Windows Command Prompt and Apache Spark 3.5.3 with Java JDK 17.

7. Implementation / Methodology

To implement the project, we have used Apache Spark and PySpark to analyze the air quality dataset. Firstly, we have imported the CSV file to create a Spark DataFrame and explored the schema and the number of records. We have converted the columns with pollutant measurements to numeric values and dropped the records that contain missing values of pollutants. After this process, 3,381 records have been left for analysis.

Secondly, we have used the StringIndexer class for converting the pollutant categories to numeric indices. Then, the VectorAssembler class was used for creating a feature vector of latitude, longitude, and pollutant index. The data was split into training and testing sets in an 80/20 ratio. We have used Linear Regression model to predict average pollutant values and Decision Tree and Random Forest classifiers to predict the categories of pollutant records defined by the project. For evaluating the regression and classification models, RMSE, MAE, and R^2 have been used for the regression model, while accuracy and F1-score have been used for the classification models.

Finally, we have generated bar plots, scatterplots, box plots, and correlation heatmap with Matplotlib and saved them as images.

8. Complete Source Code

This is the source code for implementing the Air Quality Analysis and Prediction project in Apache Spark and PySpark. This code covers data import, data cleaning, feature generation, machine learning models training, models evaluation, and data visualization.

```
# 1. LOADING THE AIR QUALITY DATASET

file_path =
r"C:\Users\bodde\OneDrive\Desktop\INT315_Air_Quality_Project\data\Air Quality
dataset.csv"

df = spark.read.csv(file_path, header=True, inferSchema=True)

print("Total rows:", df.count())

df.printSchema()

#  2. DISPLAYING POLLUTANT DATA

df.select("pollutant_min", "pollutant_max", "pollutant_avg").show(10,
truncate=False)

# 3. CONVERTING POLLUTANT COLUMNS INTO NUMERIC DATA TYPES

from pyspark.sql.functions import col

df = df.withColumn("pollutant_min", col("pollutant_min").cast("double"))

df = df.withColumn("pollutant_max", col("pollutant_max").cast("double"))

df = df.withColumn("pollutant_avg", col("pollutant_avg").cast("double"))

df.printSchema()

# 4. CHECKING MISSING VALUES

from pyspark.sql.functions import col, sum, when

df.select([

      sum(when(col(c).isNull(), 1).otherwise(0)).alias(c)

      for c in ["pollutant_min", "pollutant_max", "pollutant_avg"]

]).show()

# 5. DATA CLEANING AND PREPROCESSING

df = df.dropna(

   subset=["pollutant_min", "pollutant_max", "pollutant_avg"])

print("Rows after removing missing values:", df.count())
```

```
# 6. ANALYZING POLLUTANT DISTRIBUTION

df.groupBy("pollutant_id").count().show()

# 7. CONVERTING POLLUTANT CATEGORIES INTO NUMERICAL VALUES

from pyspark.ml.feature import StringIndexer

indexer = StringIndexer(inputCol="pollutant_id",outputCol="pollutant_index")

indexer_model = indexer.fit(df)

df = indexer_model.transform(df)

df.select("pollutant_id", "pollutant_index").show(10)

# 8. FEATURE SELECTION AND VECTOR ASSEMBLY

from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(inputCols=["latitude", "longitude",
"pollutant_index"],outputCol="features")

df = assembler.transform(df)

df.select("features", "pollutant_avg").show(10, truncate=False)

# 9. SPLITTING DATA INTO TRAINING AND TESTING SETS

train_data, test_data = df.randomSplit([0.8, 0.2], seed=42)

print("Training rows:", train_data.count())

print("Testing rows:", test_data.count())

# 10. BUILDING THE LINEAR REGRESSION MODEL

from pyspark.ml.regression import LinearRegression

lr = LinearRegression(featuresCol="features",labelCol="pollutant_avg")

lr_model = lr.fit(train_data)

predictions_lr = lr_model.transform(test_data)

predictions_lr.select("pollutant_avg","prediction").show(10, truncate=False)

# 11. EVALUATING THE LINEAR REGRESSION MODEL

from pyspark.ml.evaluation import RegressionEvaluator

rmse_evaluator =
```

```
RegressionEvaluator(labelCol="pollutant_avg",predictionCol="prediction",metricNa
me="rmse")

mae_evaluator =
RegressionEvaluator(labelCol="pollutant_avg",predictionCol="prediction",metricNa
me="mae")

r2_evaluator =
RegressionEvaluator(labelCol="pollutant_avg",predictionCol="prediction",metricNa
me="r2")

print("RMSE:", rmse_evaluator.evaluate(predictions_lr))

print("MAE:", mae_evaluator.evaluate(predictions_lr))

print("R2 Score:", r2_evaluator.evaluate(predictions_lr))

# 12. CREATING AIR QUALITY CLASSIFICATION LABELS

from pyspark.sql.functions import when, col

df = df.withColumn("air_quality",when(col("pollutant_avg") <= 50,
0).when(col("pollutant_avg") <= 100, 1).otherwise(2))

df.groupBy("air_quality").count().orderBy("air_quality").show()

# 13. PREPARING DATA FOR CLASSIFICATION

classification_train, classification_test = df.randomSplit([0.8, 0.2],seed=42)

print("Training rows:", classification_train.count())

print("Testing rows:", classification_test.count())

# 14. BUILDING THE DECISION TREE CLASSIFIER

from pyspark.ml.classification import DecisionTreeClassifier

dt = DecisionTreeClassifier(    featuresCol="features",    labelCol="air_quality",
seed=42)

dt_model = dt.fit(classification_train)

predictions_dt = dt_model.transform(classification_test)

predictions_dt.select("air_quality", "prediction", "probability").show(10,
truncate=False)

# 15. EVALUATING THE DECISION TREE CLASSIFIER

from pyspark.ml.evaluation import MulticlassClassificationEvaluator
```

```
accuracy_evaluator = MulticlassClassificationEvaluator(labelCol="air_quality",
predictionCol="prediction", metricName="accuracy")

f1_evaluator = MulticlassClassificationEvaluator(labelCol="air_quality",
predictionCol="prediction", metricName="f1")

print("Decision Tree Accuracy:", accuracy_evaluator.evaluate(predictions_dt))

print("Decision Tree F1 Score:", f1_evaluator.evaluate(predictions_dt))

# 16. BUILDING THE RANDOM FOREST CLASSIFIER

from pyspark.ml.classification import RandomForestClassifier

rf = RandomForestClassifier(featuresCol="features", labelCol="air_quality",
numTrees=100, seed=42)

rf_model = rf.fit(classification_train)

predictions_rf = rf_model.transform(classification_test)

predictions_rf.select( "air_quality", "prediction", "probability").show(10,
truncate=False)

predictions_rf.select("air_quality", "prediction", "probability").show(10,
truncate=False)

print("Random Forest Accuracy:", accuracy_evaluator.evaluate(predictions_rf))

print("Random Forest F1 Score:", f1_evaluator.evaluate(predictions_rf))

# 18. VISUALIZING POLLUTANT DISTRIBUTION USING A BAR CHART

pollutant_counts = df.groupBy("pollutant_id").count()

pollutant_counts.show()

import matplotlib.pyplot as plt

pollutant_pdf = pollutant_counts.toPandas()

plt.figure(figsize=(9, 5))

plt.bar(pollutant_pdf["pollutant_id"], pollutant_pdf["count"])

plt.title("Number of Records by Pollutant Type")

plt.xlabel("Pollutant Type")

plt.ylabel("Number of Records")

plt.tight_layout()
```

```
plt.savefig(r"C:\Users\bodde\OneDrive\Desktop\INT315_Air_Quality_Project\visualizations\pollutant_bar_chart.png")

plt.show()

# 19. VISUALIZING POLLUTANT VALUES USING A SCATTER PLOT

scatter_data = df.select("pollutant_min", "pollutant_max").toPandas()

import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.scatter(scatter_data["pollutant_min"], scatter_data["pollutant_max"], alpha=0.6)

plt.title("Minimum vs Maximum Pollutant Levels")

plt.xlabel("Minimum Pollutant Level")

plt.ylabel("Maximum Pollutant Level")

plt.tight_layout()

plt.savefig(

r"C:\Users\bodde\OneDrive\Desktop\INT315_Air_Quality_Project\visualizations\pollutant_scatter_plot.png")

plt.show()

# 20. ANALYZING POLLUTANT VALUES USING A BOX PLOT

box_data = df.select("pollutant_id", "pollutant_avg").toPandas()

plt.figure(figsize=(10, 6))

box_data.boxplot(column="pollutant_avg", by="pollutant_id")

plt.title("Average Pollutant Levels by Pollutant Type")

plt.xlabel("Pollutant Type")

plt.ylabel("Average Pollutant Level")

plt.tight_layout()

plt.savefig(r"C:\Users\bodde\OneDrive\Desktop\INT315_Air_Quality_Project\visualizations\pollutant_box_plot.png")

plt.show()
```

```
# 21. GENERATING A CORRELATION HEATMAP

from pyspark.ml.stat import Correlation

from pyspark.ml.feature import VectorAssembler

numeric_cols = ["latitude", "longitude", "pollutant_min", "pollutant_max",
"pollutant_avg"]

corr_assembler = VectorAssembler(inputCols=numeric_cols,
outputCol="corr_features")

corr_df = corr_assembler.transform(df.select(*numeric_cols).dropna())

corr_matrix = Correlation.corr(corr_df, "corr_features", "pearson").head()[0]

corr_values = corr_matrix.toArray()

print(corr_values)

# 22. DISPLAYING THE CORRELATION HEATMAP

import pandas as pd

import matplotlib.pyplot as plt

corr_pdf = pd.DataFrame(    corr_values,    index=numeric_cols,
columns=numeric_cols)

plt.figure(figsize=(9, 7))

plt.imshow(corr_pdf, vmin=-1, vmax=1, aspect="auto")

plt.colorbar(label="Pearson Correlation")

plt.xticks(range(len(numeric_cols)), numeric_cols, rotation=45, ha="right")

plt.yticks(range(len(numeric_cols)), numeric_cols)

plt.title("Correlation Heatmap of Air Quality Data")

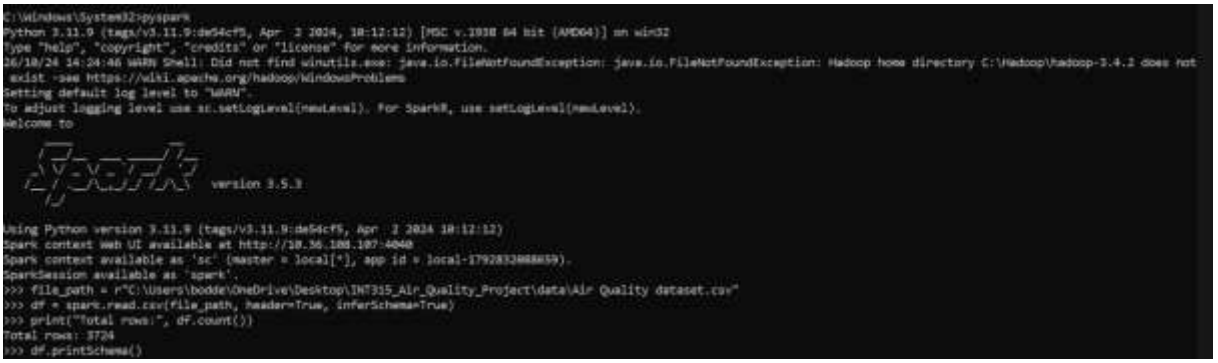
plt.tight_layout()

plt.savefig(r"C:\Users\bodde\OneDrive\Desktop\INT315_Air_Quality_Project\visual
izations\correlation_heatmap.png")

plt.show()
```

9. Execution Screenshots

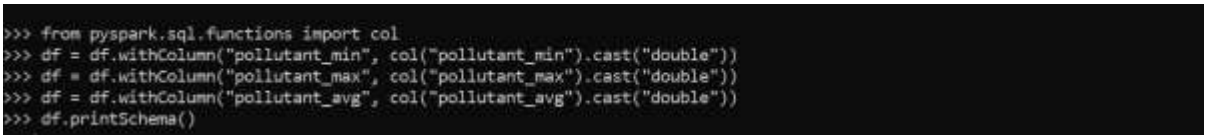
1. Loading the Air Quality Dataset in PySpark



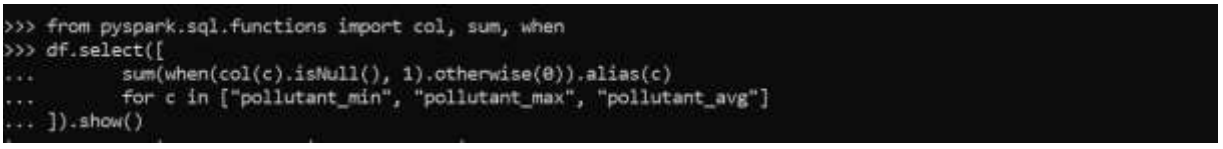
2. Displaying Pollutant Measurement Data



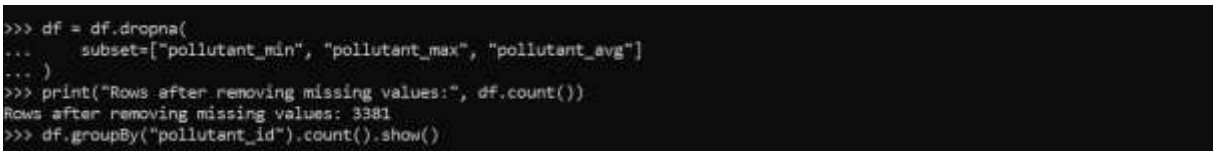
3. Converting Pollutant Columns into Numeric Data Types



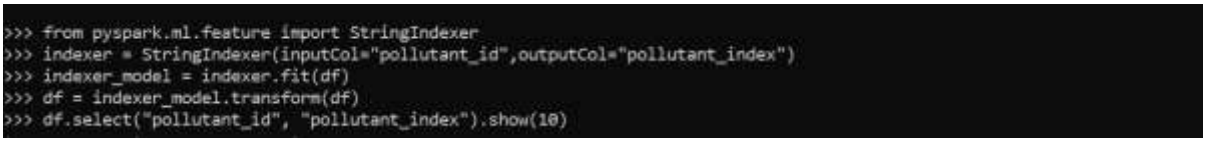
4. Checking Missing Values in the Dataset



5. Removing Missing Values and Cleaning Data



6. Converting Pollutant Categories into Numerical Values



7. Feature Selection and Vector Assembly

```
>>> from pyspark.ml.feature import VectorAssembler
>>> assembler = VectorAssembler(inputCols=["latitude", "longitude", "pollutant_index"],outputCol="features")
>>> df = assembler.transform(df)
>>> df.select("features", "pollutant_avg").show(10, truncate=False)
```

8. Splitting Data and Training the Linear Regression Model

```
>>> train_data, test_data = df.randomSplit([0.8, 0.2], seed=42)
>>> print("Training rows:", train_data.count())
Training rows: 2756
>>> print("Testing rows:", test_data.count())
Testing rows: 625
>>> from pyspark.ml.regression import LinearRegression
>>> lr = LinearRegression(featuresCol="features",labelCol="pollutant_avg")
>>> lr_model = lr.fit(train_data)
26/10/24 14:43:54 WARN Instrumentation: [ce6ec84a] regParam is zero, which might cause numerical instability and overfitting.
26/10/24 14:43:54 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.JNIBLAS
26/10/24 14:43:54 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.VectorBLAS
26/10/24 14:43:54 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.lapack.JNILAPACK
>>> predictions_lr = lr_model.transform(test_data)
>>> predictions_lr.select("pollutant_avg","prediction").show(10, truncate=False)
```

9. Evaluating the Linear Regression Model

```
>>> from pyspark.ml.evaluation import RegressionEvaluator
>>> rmse_evaluator = RegressionEvaluator(labelCol="pollutant_avg",predictionCol="prediction",metricName="rmse")
>>> mae_evaluator = RegressionEvaluator(labelCol="pollutant_avg",predictionCol="prediction",metricName="mae")
>>> r2_evaluator = RegressionEvaluator(labelCol="pollutant_avg",predictionCol="prediction",metricName="r2")
>>> print("RMSE:", rmse_evaluator.evaluate(predictions_lr))
RMSE: 53.6984841697415
>>> print("MAE:", mae_evaluator.evaluate(predictions_lr))
MAE: 39.78747313439715
>>> print("R2 Score:", r2_evaluator.evaluate(predictions_lr))
R2 Score: 0.04868716569720122
>>> from pyspark.sql.functions import when, col
>>> df = df.withColumn("air_quality",when(col("pollutant_avg") <= 50, 0).when(col("pollutant_avg") <= 100, 1).otherwise(2))
>>> df.groupBy("air_quality").count().orderBy("air_quality").show()
```

10. Creating Air Quality Classification Labels

```
>>> classification_train, classification_test = df.randomSplit([0.8, 0.2],seed=42)
>>> print("Training rows:", classification_train.count())
Training rows: 2756
>>> print("Testing rows:", classification_test.count())
Testing rows: 625
>>> from pyspark.ml.classification import DecisionTreeClassifier
>>> dt = DecisionTreeClassifier(    featuresCol="features",    labelCol="air_quality",    seed=42)
>>> dt_model = dt.fit(classification_train)
>>> predictions_dt = dt_model.transform(classification_test)
>>> predictions_dt.select("air_quality", "prediction", "probability").show(10, truncate=False)
```

11. Training the Decision Tree Classifier

```
>>> from pyspark.ml.evaluation import MulticlassClassificationEvaluator
>>> accuracy_evaluator = MulticlassClassificationEvaluator(labelCol="air_quality", predictionCol="prediction", metricName="accuracy")
>>> f1_evaluator = MulticlassClassificationEvaluator(labelCol="air_quality", predictionCol="prediction", metricName="f1")
>>> print("Decision Tree Accuracy:", accuracy_evaluator.evaluate(predictions_dt))
Decision Tree Accuracy: 0.792
>>> print("Decision Tree F1 Score:", f1_evaluator.evaluate(predictions_dt))
Decision Tree F1 Score: 0.7779233327402688
>>> from pyspark.ml.classification import RandomForestClassifier
>>> rf = RandomForestClassifier(featuresCol="features", labelCol="air_quality", numTrees=100, seed=42)
>>> rf_model = rf.fit(classification_train)
>>> predictions_rf = rf_model.transform(classification_test)
>>> predictions_rf.select(    "air_quality",    "prediction",    "probability").show(10, truncate=False)
```

12. Displaying Random Forest Predictions and Evaluation Results

```
>>> print("Random Forest Accuracy:", accuracy_evaluator.evaluate(predictions_rf))
Random Forest Accuracy: 0.7984
>>> print("Random Forest F1 Score:", f1_evaluator.evaluate(predictions_rf))
Random Forest F1 Score: 0.7837897118994501
>>> pollutant_counts = df.groupBy("pollutant_id").count()
>>> pollutant_counts.show()
```

13. Generating the Pollutant Distribution Bar Chart

```
>>> import matplotlib.pyplot as plt
>>> pollutant_pdf = pollutant_counts.toPandas()
>>> plt.figure(figsize=(9, 5))
<Figure size 900x500 with 0 Axes>
>>> plt.bar(pollutant_pdf["pollutant_id"], pollutant_pdf["count"])
<BarContainer object of 7 artists>
>>> plt.title("Number of Records by Pollutant Type")
Text(0.5, 1.0, 'Number of Records by Pollutant Type')
>>> plt.xlabel("Pollutant Type")
Text(0.5, 0, 'Pollutant Type')
>>> plt.ylabel("Number of Records")
Text(0, 0.5, 'Number of Records')
>>> plt.tight_layout()
>>> plt.savefig(r"C:\Users\bodde\OneDrive\Desktop\INT315_Air_Quality_Project\visualizations\pollutant_bar_chart.png")
>>> plt.show()
```

14. Generating the Pollutant Scatter Plot

```
>>> scatter_data = df.select("pollutant_min", "pollutant_max").toPandas()
>>> import matplotlib.pyplot as plt
>>> plt.figure(figsize=(8, 5))
<Figure size 800x500 with 0 Axes>
>>> plt.scatter(scatter_data["pollutant_min"], scatter_data["pollutant_max"], alpha=0.6)
<matplotlib.collections.PathCollection object at 0x0000020BB51E3810>
>>> plt.title("Minimum vs Maximum Pollutant Levels")
Text(0.5, 1.0, 'Minimum vs Maximum Pollutant Levels')
>>> plt.xlabel("Minimum Pollutant Level")
Text(0.5, 0, 'Minimum Pollutant Level')
>>> plt.ylabel("Maximum Pollutant Level")
Text(0, 0.5, 'Maximum Pollutant Level')
>>> plt.tight_layout()
>>> plt.savefig(
... r"C:\Users\bodde\OneDrive\Desktop\INT315_Air_Quality_Project\visualizations\pollutant_scatter_plot.png")
>>> plt.show()
```

15. Generating the Pollutant Box Plot

```
>>> box_data.boxplot(column="pollutant_avg", by="pollutant_id")
<Axes: title={'center': 'pollutant_avg'}, xlabel='pollutant_id'>
>>> plt.title("Average Pollutant Levels by Pollutant Type")
Text(0.5, 1.0, 'Average Pollutant Levels by Pollutant Type')
>>> plt.xlabel("Pollutant Type")
Text(0.5, 0, 'Pollutant Type')
>>> plt.ylabel("Average Pollutant Level")
Text(0, 0.5, 'Average Pollutant Level')
>>> plt.tight_layout()
>>> plt.savefig(r"C:\Users\bodde\OneDrive\Desktop\INT315_Air_Quality_Project\visualizations\pollutant_box_plot.png")
>>> plt.show()
```

16. Calculating Correlations Between Numerical Features

```
>>> from pyspark.ml.stat import Correlation
>>> from pyspark.ml.feature import VectorAssembler
>>> numeric_cols = ["latitude", "longitude", "pollutant_min", "pollutant_max", "pollutant_avg"]
>>> corr_assembler = VectorAssembler(inputCols=numeric_cols, outputCol="corr_features")
>>> corr_df = corr_assembler.transform(df.select(*numeric_cols).dropna())
>>> corr_matrix = Correlation.corr(corr_df, "corr_features", "pearson").head()[0]
>>> corr_values = corr_matrix.toArray()
>>> print(corr_values)
```

17. Generating the Correlation Heatmap

```
>>> import pandas as pd
>>> import matplotlib.pyplot as plt
>>> corr_pdf = pd.DataFrame(    corr_values,    index=numeric_cols,    columns=numeric_cols)
>>> plt.figure(figsize=(8, 7))
<Figure size 800x700 with 0 Axes>
>>> plt.imshow(corr_pdf, vmin=-1, vmax=1, aspect="auto")
<matplotlib.image.AxesImage object at 0x0000020BB55F8E50>
>>> plt.colorbar(label="Pearson Correlation")
<matplotlib.colorbar.Colorbar object at 0x0000020BB5736710>
>>> plt.xticks(range(len(numeric_cols)), numeric_cols, rotation=45, ha="right")
([<matplotlib.axis.XTick object at 0x0000020BB55C7310>, <matplotlib.axis.XTick object at 0x0000020BB56A3490>, <matplotlib.axis.XTick object at 0x0000020BB56BF290>, <matplotlib.axis.XTick object at 0x0000020BB56A8D10>, <matplotlib.axis.XTick object at 0x0000020BB564CBD0>], [Text(0, 0, 'latitude'), Text(1, 0, 'longitude'), Text(2, 0, 'pollutant_min'), Text(3, 0, 'pollutant_max'), Text(4, 0, 'pollutant_avg')])
>>> plt.yticks(range(len(numeric_cols)), numeric_cols)
([<matplotlib.axis.YTick object at 0x0000020BB55CBD50>, <matplotlib.axis.YTick object at 0x0000020BB5737C50>, <matplotlib.axis.YTick object at 0x0000020BB5690DD0>, <matplotlib.axis.YTick object at 0x0000020BB5689D90>, <matplotlib.axis.YTick object at 0x0000020BB56E9090>], [Text(0, 0, 'latitude'), Text(0, 1, 'longitude'), Text(0, 2, 'pollutant_min'), Text(0, 3, 'pollutant_max'), Text(0, 4, 'pollutant_avg')])
>>> plt.title("Correlation Heatmap of Air Quality Data")
Text(0.5, 1.0, 'Correlation Heatmap of Air Quality Data')
>>> plt.tight_layout()
>>> plt.savefig(r"C:\Users\bodde\OneDrive\Desktop\INT315_Air_Quality_Project\visualizations\correlation_heatmap.png")
>>> plt.show()
```

10. Output/Result Screenshots

1. Displaying the Dataset Schema

```
>>> df.printSchema()
root
 |-- country: string (nullable = true)
 |-- state: string (nullable = true)
 |-- city: string (nullable = true)
 |-- station: string (nullable = true)
 |-- last_update: string (nullable = true)
 |-- latitude: double (nullable = true)
 |-- longitude: double (nullable = true)
 |-- pollutant_id: string (nullable = true)
 |-- pollutant_min: string (nullable = true)
 |-- pollutant_max: string (nullable = true)
 |-- pollutant_avg: string (nullable = true)
```

2. Preview of Pollutant Measurements

```
>>> df.select(
...     "pollutant_min",
...     "pollutant_max",
...     "pollutant_avg"
... ).show(10, truncate=False)
+-------------+-------------+-------------+
|pollutant_min|pollutant_max|pollutant_avg|
+-------------+-------------+-------------+
|0            |5            |2            |
|17           |41           |31           |
|49           |169          |111          |
|0            |13           |5            |
|NA           |NA           |NA           |
|43           |59           |53           |
|4            |20           |12           |
|19           |85           |51           |
|37           |54           |44           |
|21           |25           |23           |
+-------------+-------------+-------------+
only showing top 10 rows
```

3. Dataset Schema After Numeric Conversion

```
>>> from pyspark.sql.functions import col
>>> df = df.withColumn("pollutant_min", col("pollutant_min").cast("double"))
>>> df = df.withColumn("pollutant_max", col("pollutant_max").cast("double"))
>>> df = df.withColumn("pollutant_avg", col("pollutant_avg").cast("double"))
>>> df.printSchema()
root
 |-- country: string (nullable = true)
 |-- state: string (nullable = true)
 |-- city: string (nullable = true)
 |-- station: string (nullable = true)
 |-- last_update: string (nullable = true)
 |-- latitude: double (nullable = true)
 |-- longitude: double (nullable = true)
 |-- pollutant_id: string (nullable = true)
 |-- pollutant_min: double (nullable = true)
 |-- pollutant_max: double (nullable = true)
 |-- pollutant_avg: double (nullable = true)
```

4. Missing Value Counts

```
>>> from pyspark.sql.functions import col, sum, when
>>> df.select([
...         sum(when(col(c).isNull(), 1).otherwise(0)).alias(c)
...         for c in ["pollutant_min", "pollutant_max", "pollutant_avg"]
... ]).show()
+-------------+-------------+-------------+
|pollutant_min|pollutant_max|pollutant_avg|
+-------------+-------------+-------------+
|          343|          343|          343|
+-------------+-------------+-------------+
```

5. Number of Records by Pollutant Type

```
>>> df.groupBy("pollutant_id").count().show()
+------------+-----+
|pollutant_id|count|
+------------+-----+
|       OZONE|  507|
|        PM10|  484|
|         NO2|  491|
|         NH3|  439|
|          CO|  511|
|       PM2.5|  482|
|         SO2|  467|
+------------+-----+
```

6. Numerical Encoding of Pollutant Categories

```
>>> df.select("pollutant_id", "pollutant_index").show(10)
+------------+---------------+
|pollutant_id|pollutant_index|
+------------+---------------+
|       OZONE|            1.0|
|          CO|            0.0|
|        PM10|            3.0|
|         SO2|            5.0|
|          CO|            0.0|
|         SO2|            5.0|
|       OZONE|            1.0|
|          CO|            0.0|
|       PM2.5|            4.0|
|        PM10|            3.0|
+------------+---------------+
only showing top 10 rows
```

7. Assembled Feature Vectors

```
>>> df.select("features", "pollutant_avg").show(10, truncate=False)
+----------------------------+-------------+
|features                    |pollutant_avg|
+----------------------------+-------------+
|[24.82827,92.79525,1.0]     |2.0          |
|[26.987634,94.636574,0.0]   |31.0         |
|[26.146529,87.454184,3.0]   |111.0        |
|[26.146529,87.454184,5.0]   |5.0          |
|[25.5626095,84.663264,0.0]  |53.0         |
|[24.75746,84.366208,5.0]    |12.0         |
|[24.75746,84.366208,1.0]    |51.0         |
|[25.42742023,86.13886079,0.0]|44.0         |
|[26.80365,84.51954,4.0]     |23.0         |
|[26.80365,84.51954,3.0]     |21.0         |
+----------------------------+-------------+
only showing top 10 rows
```

8. Linear Regression Prediction Results

```
>>> predictions_lr.select("pollutant_avg","prediction").show(10, truncate=False)
+-------------+-----------------+
|pollutant_avg|prediction       |
+-------------+-----------------+
|20.0         |28.644789788584205|
|14.0         |23.82109698240003 |
|5.0          |22.16675815417649 |
|15.0         |23.774655756211217|
|67.0         |18.859621091353603|
|11.0         |28.20920400263465 |
|8.0          |21.85240636279724 |
|35.0         |29.166062417551743|
|6.0          |24.568985231963666|
|32.0         |31.000575640102568|
+-------------+-----------------+
only showing top 10 rows
```

9. Distribution of Air Quality Classification Labels

```
>>> df.groupBy("air_quality").count().orderBy("air_quality").show()
+-----------+-----+
|air_quality|count|
+-----------+-----+
|          0| 2261|
|          1|  641|
|          2|  479|
+-----------+-----+
```

10. Decision Tree Classification Predictions

```
>>> predictions_dt.select("air_quality", "prediction", "probability").show(10, truncate=False)
+-----------+----------+-----------------------------------------------------------+
|air_quality|prediction|probability                                                |
+-----------+----------+-----------------------------------------------------------+
|0          |0.0       |[0.8511363636363637,0.13977272727272727,0.009090909090909]|
|0          |0.0       |[0.9892328398384926,0.010767160161507403,0.0]              |
|0          |0.0       |[0.9892328398384926,0.010767160161507403,0.0]              |
|0          |0.0       |[0.9892328398384926,0.010767160161507403,0.0]              |
|1          |0.0       |[0.8076923076923077,0.15384615384615385,0.038461538461538464]|
|0          |0.0       |[0.8511363636363637,0.13977272727272727,0.009090909090909]|
|0          |0.0       |[0.9892328398384926,0.010767160161507403,0.0]              |
|0          |0.0       |[0.8511363636363637,0.13977272727272727,0.009090909090909]|
|0          |0.0       |[0.9892328398384926,0.010767160161507403,0.0]              |
|0          |0.0       |[0.8511363636363637,0.13977272727272727,0.009090909090909]|
+-----------+----------+-----------------------------------------------------------+
only showing top 10 rows
```

11. Random Forest Classification Predictions

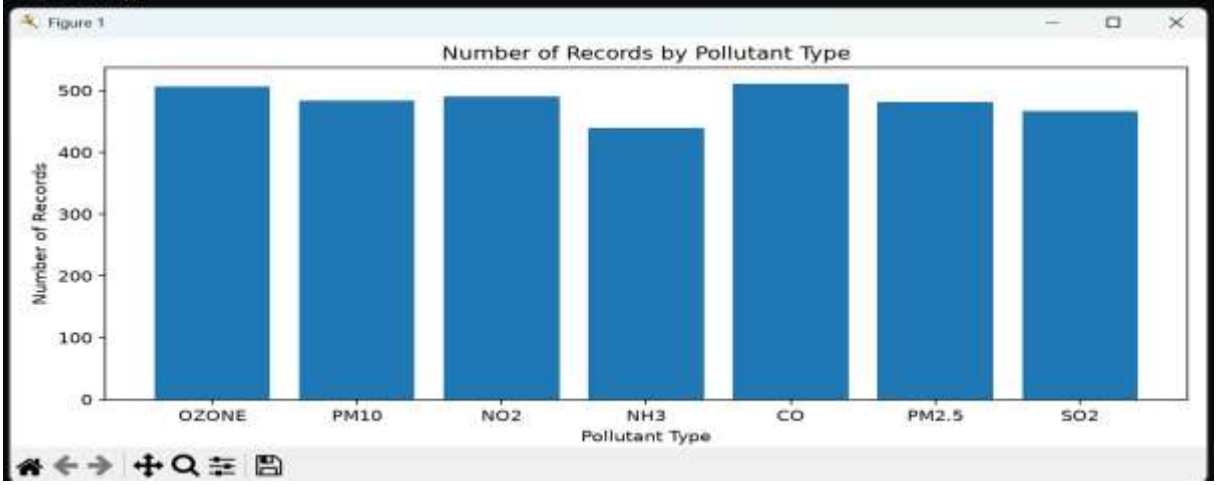
```
>>> predictions_rf.select("air_quality", "prediction", "probability").show(10, truncate=False)
+-----------+----------+-----------------------------------------------------------+
|air_quality|prediction|probability                                                |
+-----------+----------+-----------------------------------------------------------+
|0          |0.0       |[0.899892530205483,0.09288391621957236,0.007223553574879353]|
|0          |0.0       |[0.9271753987480629,0.06756838449909723,0.005256296752839847]|
|0          |0.0       |[0.9266856513658932,0.06801380086794057,0.005300547766166181]|
|0          |0.0       |[0.9266856513658932,0.06801380086794057,0.005300547766166181]|
|1          |0.0       |[0.7161491834278818,0.2391769541420524,0.04467386243006756]|
|0          |0.0       |[0.899892530205483,0.09288391621957236,0.007223553574879353]|
|0          |0.0       |[0.9271753987480629,0.06756838449909723,0.005256296752839847]|
|0          |0.0       |[0.8986662796168282,0.09408869483824638,0.007245025544925506]|
|0          |0.0       |[0.9266856513658932,0.06801380086794057,0.005300547766166181]|
|0          |0.0       |[0.899021806757968,0.09377996030595598,0.007198232936076081]|
+-----------+----------+-----------------------------------------------------------+
only showing top 10 rows
```

12. Pollutant Record Counts

```
>>> pollutant_counts.show()
+------------+-----+
|pollutant_id|count|
+------------+-----+
|       OZONE|  507|
|        PM10|  484|
|         NO2|  491|
|         NH3|  439|
|          CO|  511|
|       PM2.5|  482|
|         SO2|  467|
+------------+-----+
```

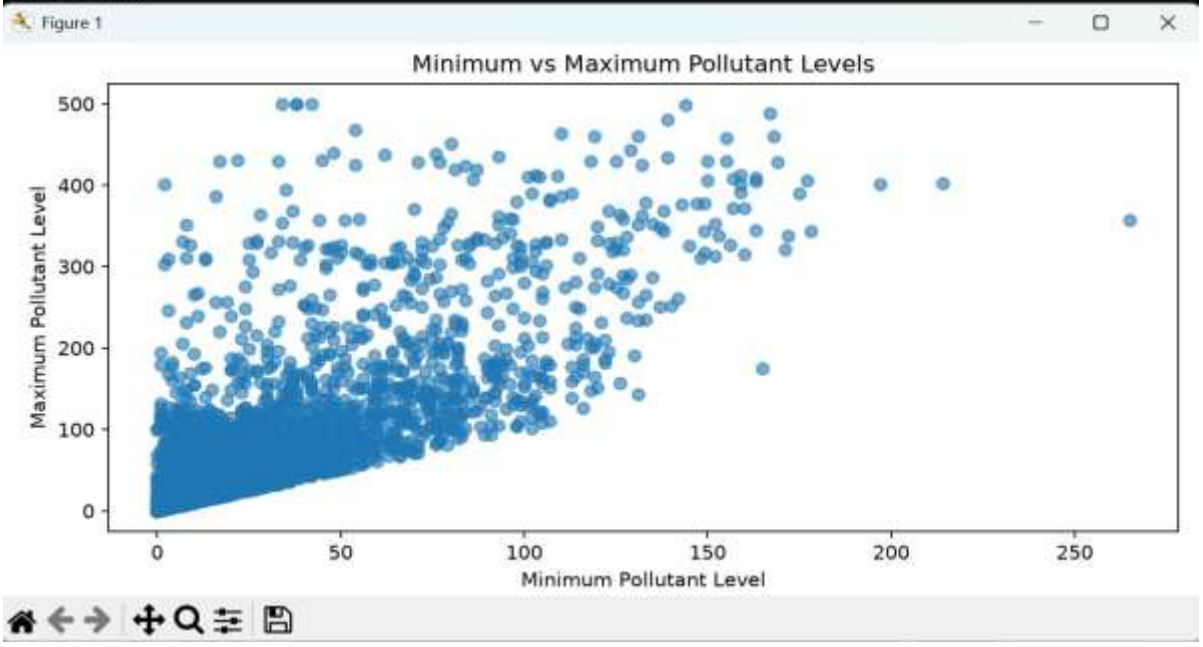
13. Bar Chart of Records by Pollutant Type

```
>>> import matplotlib.pyplot as plt
>>> pollutant_pdf = pollutant_counts.toPandas()
>>> plt.figure(figsize=(9, 5))
<Figure size 900x500 with 0 Axes>
>>> plt.bar(pollutant_pdf["pollutant_id"], pollutant_pdf["count"])
<BarContainer object of 7 artists>
>>> plt.title("Number of Records by Pollutant Type")
Text(0.5, 1.0, 'Number of Records by Pollutant Type')
>>> plt.xlabel("Pollutant Type")
Text(0.5, 0, 'Pollutant Type')
>>> plt.ylabel("Number of Records")
Text(0, 0.5, 'Number of Records')
>>> plt.tight_layout()
>>> plt.savefig(r"C:\Users\bodde\OneDrive\Desktop\INT315_Air_Quality_Project\visualizations\pollutant_bar_chart.png")
>>> plt.show()
```



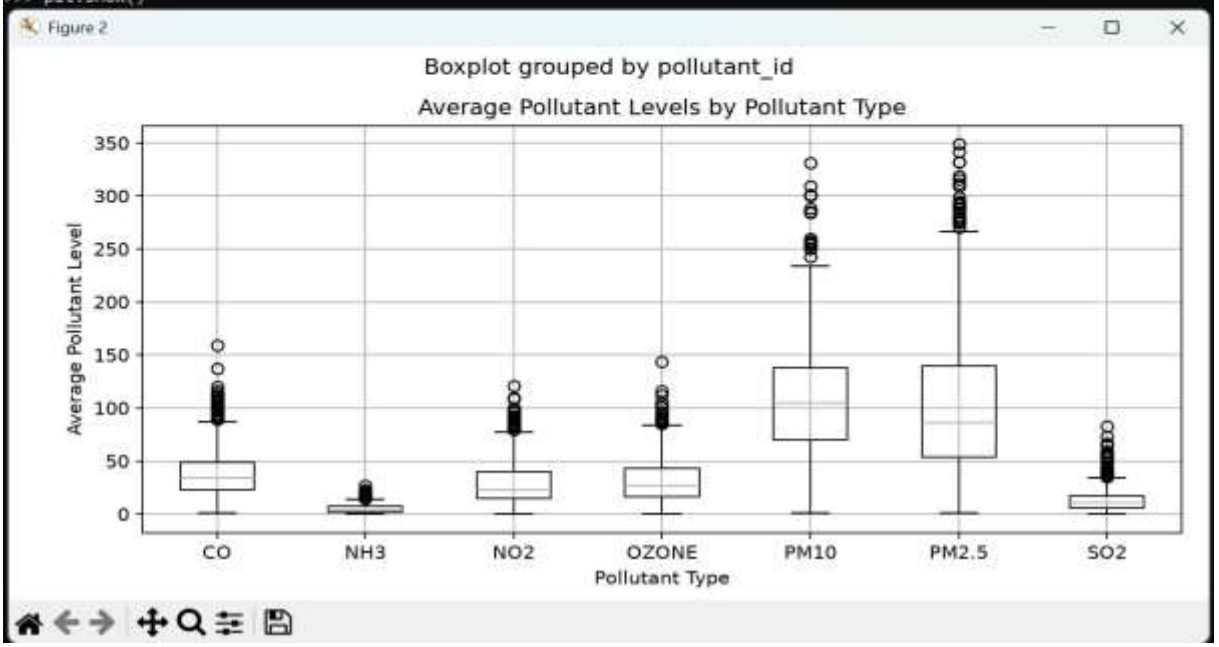
14. Scatter Plot of Minimum and Maximum Pollutant Levels

```
>>> scatter_data = df.select("pollutant_min", "pollutant_max").toPandas()
>>> import matplotlib.pyplot as plt
>>> plt.figure(figsize=(8, 5))
<Figure size 800x500 with 0 Axes>
>>> plt.scatter(scatter_data["pollutant_min"], scatter_data["pollutant_max"], alpha=0.6)
<matplotlib.collections.PathCollection object at 0x00000208851E3810>
>>> plt.title("Minimum vs Maximum Pollutant Levels")
Text(0.5, 1.0, 'Minimum vs Maximum Pollutant Levels')
>>> plt.xlabel("Minimum Pollutant Level")
Text(0.5, 0, 'Minimum Pollutant Level')
>>> plt.ylabel("Maximum Pollutant Level")
Text(0, 0.5, 'Maximum Pollutant Level')
>>> plt.tight_layout()
>>> plt.savefig(
... r"C:\Users\bodde\OneDrive\Desktop\INT315_Air_Quality_Project\visualizations\pollutant_scatter_plot.png")
>>> plt.show()
```

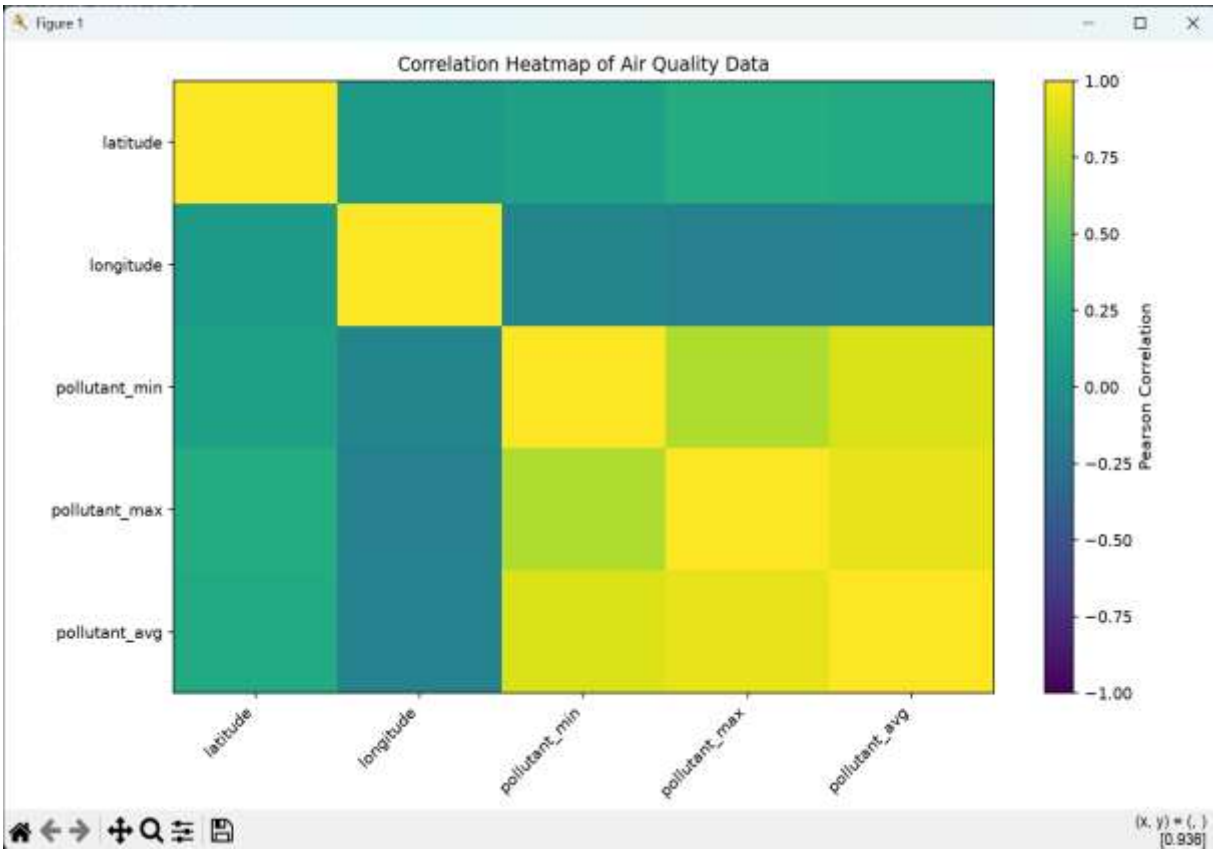


15. Box Plot of Average Pollutant Levels by Pollutant Type

```
>>> box_data = df.select("pollutant_id", "pollutant_avg").toPandas()
>>> plt.figure(figsize=(10, 6))
<Figure size 1000x600 with 0 Axes>
>>> box_data.boxplot(column="pollutant_avg", by="pollutant_id")
<Axes: title={'center': 'pollutant_avg'}, xlabel='pollutant_id'>
>>> plt.title("Average Pollutant Levels by Pollutant Type")
Text(0.5, 1.0, 'Average Pollutant Levels by Pollutant Type')
>>> plt.xlabel("Pollutant Type")
Text(0.5, 0, 'Pollutant Type')
>>> plt.ylabel("Average Pollutant Level")
Text(0, 0.5, 'Average Pollutant Level')
>>> plt.tight_layout()
>>> plt.savefig(r"C:\Users\bodde\OneDrive\Desktop\INT315_Air_Quality_Project\visualizations\pollutant_box_plot.png")
>>> plt.show()
```



16. Correlation Heatmap of Air Quality Data



11. Result Analysis

The air quality data set was processed through Apache Spark, where missing values of pollutants were cleaned out in preprocessing. The processed data set had 3,381 records.

For Linear Regression, the RMSE is 53.70, MAE is 39.79, and R^2 value is 0.0487 which suggests that Linear Regression does not have good predictive power with chosen input variables.

For the Decision Tree algorithm, accuracy is 79.20%, and F1-score is 0.7779. For the Random Forest algorithm, accuracy is 79.84%, and F1-score is 0.7838. Hence, Random Forest outperforms Decision Tree in predicting the project-defined air quality categories.

The visuals, like the pollutant bar graph, scatter graph, box plot, and correlation heatmap, helped in the identification of the patterns of pollutants in the data set.

Conclusion

The work involved the application of Apache Spark in analyzing and predicting air pollution. The datasets were cleaned and processed through PySpark, and Linear Regression, Decision Tree, and Random Forest algorithms were applied to the data.

Among all classification algorithms, Random Forest has shown the best results in terms of accuracy (79.84%). The plots allowed gaining insight into the pollution distributions and dependencies between pollutant measurements.

On the whole, the project was a good opportunity to gain practical skills in big data processing, machine learning, and data visualization via Apache Spark. In order to improve the system in the future, other environmental factors and proper features should be considered.